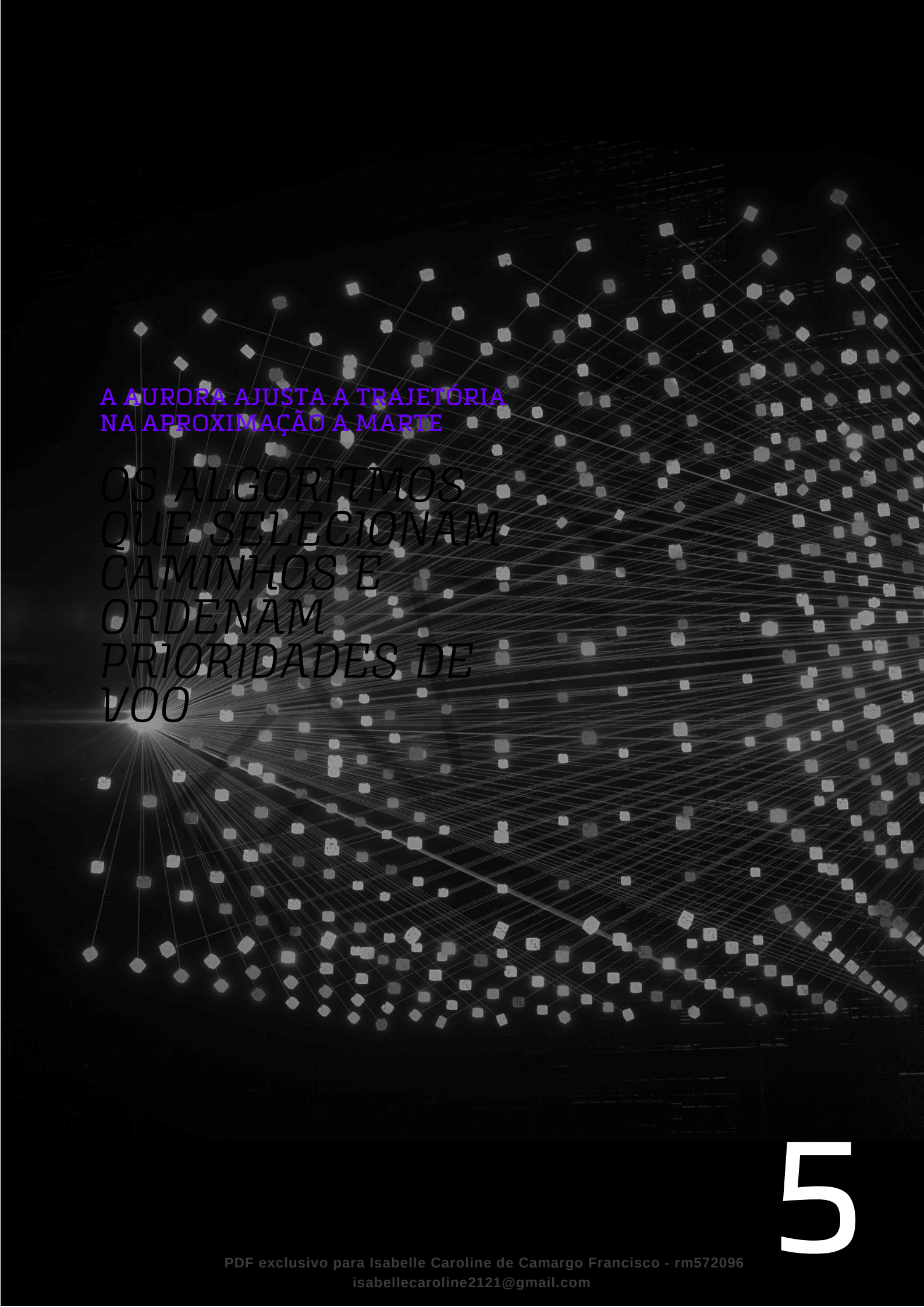




A AURORA AJUSTA A TRAJETÓRIA
NA APROXIMAÇÃO A MARTE

OS ALGORITMOS
QUE SELECIONAM
CAMINHOS E
ORDENAM
PRIORIDADES DE
VOO

5

LISTA DE FIGURAS

Figura 1 – Fluxograma da busca linear	18
Figura 2 – Fluxograma da busca binária	20
Figura 3 – Funcionamento do <i>bubble sort</i>	26
Figura 4 – Funcionamento do <i>selection sort</i>	28
Figura 5 – Funcionamento do <i>insertion sort</i>	29



LISTA DE TABELAS

Tabela 1 – Diferença entre melhor caso, pior caso e caso médio.....	11
Tabela 2 – Complexidade e operações.....	11

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de complexidade $O(1)$ em um programa em C	8
Código-fonte 2 – Exemplo de complexidade $O(\log n)$ em um programa em C	9
Código-fonte 3 – Exemplo de complexidade $O(n)$ em um programa em C	9
Código-fonte 4 – Exemplo de complexidade $O(n^2)$ em um programa em C	10
Código-fonte 5 – Exemplo de acesso a uma posição em um programa em C	14
Código-fonte 6 – Exemplo de comparação em um programa em C	14
Código-fonte 7 – Exemplo de busca linear em um programa em C	14
Código-fonte 8 – Exemplo de um passo fundamental da busca binária em um programa em C	15
Código-fonte 9 – Exemplo de retorno da posição em um programa em C	15
Código-fonte 10 – Exemplo de busca encadeada em um programa em C	16
Código-fonte 11 – Exemplo de código da busca linear em um programa em C	19
Código-fonte 12 – Exemplo de código da busca binária em um programa em C	22
Código-fonte 13 – Exemplo de código de ordenação <i>bubble sort</i> em um programa em C	27
Código-fonte 14 – Exemplo de código de ordenação <i>selection sort</i> em um programa em C	28
Código-fonte 15 – Exemplo de código de ordenação <i>insertion sort</i> em um programa em C	30
Código-fonte 16 – Exemplo de código de ordenação <i>shell sort</i> em um programa em C	32
Código-fonte 17 – Exemplo de código de ordenação <i>comb sort</i> em um programa em C	33
Código-fonte 18 – Exemplo de código de ordenação <i>merge sort</i> em um programa em C	35
Código-fonte 19 – Exemplo de código de ordenação <i>quick sort</i> em um programa em C	36
Código-fonte 19 – Exemplo de código de ordenação <i>heap sort</i> em um programa em C	37

SUMÁRIO

ALGORITMOS DE BUSCA E ORDENAÇÃO	6
1 INTRODUÇÃO	6
1.1 Complexidade de algoritmos	7
1.1.1 Entendendo a notação Big-O	7
1.1.2 Tipos mais comuns de complexidade	8
1.1.3 Diferença entre melhor caso, pior caso e caso médio.....	10
2 FUNDAMENTOS DA BUSCA	11
2.1 O problema da busca	12
2.2 Operações fundamentais da busca.....	13
2.2.1 Operação 1 – Acesso a uma posição (<i>access operation</i>)	13
2.2.2 Operação 2 – Comparação (<i>comparison operation</i>)	14
2.2.3 Operação 3 – Iteração sobre elementos (<i>iteration operation</i>)	14
2.2.4 Operação 4 - Divisão do intervalo (<i>divide operation</i>).....	15
2.2.5 Operação 5 – Retorno da posição (<i>return operation</i>)	15
2.2.6 Operação 6 – Mudança de ponteiros / referências (<i>pointer update</i>)	15
2.3 Por que essas operações importam?	16
2.4 Tipos fundamentais de operações de busca	16
2.4.1 Busca linear.....	17
2.4.2 Busca binária.....	19
3 FUNDAMENTOS DA ORDENAÇÃO.....	23
3.1 Por que ordenar? A motivação computacional.....	24
3.2 Classificação dos algoritmos de ordenação	24
3.3 Ordenação simples	25
3.3.1 <i>Bubble sort</i>	26
3.3.2 <i>Selection sort</i>	27
3.3.3 <i>Insertion sort</i>	29
3.4 Ordenação intermediária	30
3.4.1 <i>Shell sort</i>	31
3.4.2 <i>Comb sort</i>	32
3.5. Algoritmos de ordenação eficiente ($O(n \log n)$)	33
3.5.1 <i>Merge sort</i>	34
3.5.2 <i>Quick sort</i>	35
3.5.3 <i>Heap sort</i>	36
4 CONCLUSÃO.....	38
4.1 Outras ordenações relevantes	39
REFERÊNCIAS.....	41

ALGORITMOS DE BUSCA E ORDENAÇÃO

1 INTRODUÇÃO

Os algoritmos de busca e ordenação constituem um conjunto central de ferramentas teóricas e práticas na ciência da computação. Em praticamente todas as áreas, como bancos de dados, sistemas operacionais, computação gráfica, redes de computadores, inteligência artificial e desenvolvimento de software embarcado, surge a necessidade de acessar, localizar ou reorganizar coleções de dados de maneira eficiente. Uma tarefa aparentemente simples, como localizar o registro de um cliente, ordenar uma lista de produtos ou determinar se um elemento pertence a um conjunto, pode tornar-se extremamente custosa quando aplicada sobre grandes volumes de dados ou em ambientes em que o desempenho é crítico.

Ao longo da história da computação, pesquisadores identificaram que a eficiência com que um sistema executa operações de busca e ordenação não depende apenas do hardware disponível, mas principalmente dos algoritmos adotados. De acordo com Cormen *et al.* (2009), a escolha adequada de um algoritmo pode reduzir o tempo de processamento de horas para segundos, especialmente em cenários que envolvem milhões ou bilhões de registros. Em ambientes em que existe a limitação de recursos, como sistemas embarcados, dispositivos IoT ou aplicações industriais em tempo real, a eficiência algorítmica não é apenas desejável, mas uma exigência operacional.

Ao contrário do que possa parecer, algoritmos de busca e ordenação não se limitam à organização de listas ou vetores. Segundo Sedgewick e Wayne (2011), eles influenciam diretamente o funcionamento de mecanismos de indexação, sistemas de recuperação de informação, compressão de dados, criptografia, sistemas distribuídos, caches de memória e até mesmo técnicas modernas de aprendizado de máquina. Por essa razão, compreender profundamente tais algoritmos é um passo essencial na formação de qualquer cientista da computação.

Este capítulo dedica-se a explorar os principais algoritmos de busca e ordenação utilizados na linguagem C, investigando seus princípios, características, vantagens e limitações.

Ao concluir esse conteúdo, o estudante terá adquirido não apenas a habilidade de implementar algoritmos clássicos, como busca linear, busca binária, *bubble sort*, *merge sort* ou *quick sort*, mas também a capacidade crítica de escolher o algoritmo mais apropriado para um determinado problema, justificando sua decisão com base em propriedades formais e empíricas.

1.1 Complexidade de algoritmos

Antes de iniciar o estudo sobre os algoritmos de busca e ordenação, iremos compreender a complexidade de algoritmos como um conceito fundamental para analisar e comparar métodos de busca e ordenação. De acordo com Cormen *et al.* (2009), a análise de complexidade permite estimar o comportamento de um algoritmo para entradas de tamanhos crescentes, independentemente da máquina, do compilador ou da implementação específica. Isso fornece um modelo matemático robusto para avaliar desempenho, tempo de execução e consumo de memória.

A notação mais conhecida para essa análise é a **Big-O**, utilizada para descrever o limite superior do crescimento da função de tempo de um algoritmo. Ela abstrai detalhes menores e expressa apenas o que ocorre quando o tamanho da entrada tende ao infinito.

1.1.1 Entendendo a notação Big-O

A notação Big-O expressa o tempo de execução de um algoritmo em função do tamanho de entrada n . Assim, dizemos que um algoritmo é:

$$T(n) = O(f(n))$$

Quando existe uma constante positiva c e um valor n_0 , tais que:

$$T(n) \leq c \cdot f(n) \text{ para todo } n \geq n_0$$

Em termos simples, essa notação indica como o tempo de execução cresce à medida que os dados aumentam.

1.1.2 Tipos mais comuns de complexidade

A seguir, apresentaremos as complexidades mais frequentes na área de estruturas de dados e algoritmos, representadas pelos números de (1) a (6).

1. $O(1)$ - Tempo constante: o tempo não depende do tamanho da entrada.

Exemplos típicos:

- Acessar um elemento de um vetor por índice;
- Inserção ou remoção no topo de uma pilha;
- Operações aritméticas simples.

Exemplo em C:

```
int x = vetor[5]; // acesso direto → tempo constante
```

Código-fonte 1 – Exemplo de complexidade $O(1)$ em um programa em C
Fonte: Elaborado pelo autor (2025)

Mesmo que o vetor tenha 10 ou 1 milhão de elementos, o tempo é o mesmo.

2. $O(\log n)$ - Tempo logarítmico: ocorre quando o algoritmo reduz o problema pela metade a cada passo.

Exemplos clássicos:

- Busca binária;
- Altura de árvores balanceadas;
- Métodos de divisão e conquista.

Exemplo intuitivo: para encontrar uma palavra em um dicionário físico, não viramos página por página (isso seria $O(n)$). Dividimos o livro no meio, decidimos se precisamos ir para a esquerda ou direita, e repetimos.

Em C (busca binária):


```
int meio = (inicio + fim) / 2;
```

Código-fonte 2 – Exemplo de complexidade $O(\log n)$ em um programa em C
Fonte: Elaborado pelo autor (2025)

Esse “dividir ao meio” gera a complexidade logarítmica.

3. $O(n)$ - Tempo linear: o tempo cresce proporcionalmente ao tamanho da entrada.

Exemplos:

- Busca linear;
- Percorrer uma lista encadeada;
- Contar elementos.

Código típico em C:

```
for (int i = 0; i < n; i++) {  
    if (v[i] == chave) return i;  
}
```

Código-fonte 3 – Exemplo de complexidade $O(n)$ em um programa em C
Fonte: Elaborado pelo autor (2025)

Se o vetor dobrar de tamanho, o tempo dobra.

4. $O(n \log n)$: é a combinação de dividir e percorrer.

Está presente em algoritmos eficientes de ordenação, como:

- Merge sort;
- *Quick sort* (caso médio);
- Heap sort.

Esse é o melhor desempenho médio possível para algoritmos baseados em comparação (Knuth, 1998).

5. $O(n^2)$ - Tempo quadrático: ocorre quando há dois laços aninhados:

```
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        // operação
    }
}
```

Código-fonte 4 – Exemplo de complexidade $O(n^2)$ em um programa em C
Fonte: Elaborado pelo autor (2025)

Exemplos:

- Bubble sort;
- Selection sort;
- *Insertion sort* (na maioria dos casos).

Para $n = 1.000$, isso corresponde a 1.000.000 operações, tornando-se inviável para grandes volumes.

6. Tempos exponenciais ou piores:

- $O(2^n)$ - Algoritmos ingênuos de força bruta;
- $O(n!)$ - Permutações completas (como algumas versões do problema do caixeiro viajante).

São considerados impraticáveis exceto para valores muito pequenos de entrada.

1.1.3 Diferença entre melhor caso, pior caso e caso médio

Todo algoritmo pode comportar-se de forma diferente dependendo da entrada.

Cenário	Significado	Exemplo na prática
Melhor caso	Situação mais favorável	Busca linear encontra na 1ª posição
Pior caso	Situação mais desfavorável	<i>Bubble sort</i> com vetor invertido
Caso médio	Entrada típica, usada em análises	<i>Quick sort</i> com pivô razoável

Tabela 1 – Diferença entre melhor caso, pior caso e caso médio
 Fonte: Elaborada pelo autor (2025)

Essa distinção é essencial na avaliação de desempenho. A ordem de crescimento relativa é:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

A pergunta é sempre:

“Se o número de dados aumentar, como o tempo de execução cresce?”

Supondo que existam 1000 execuções, ou seja, $n = 1000$, a tabela abaixo apresenta o tempo de operação.

Complexidade	Operações
$O(1)$	1
$O(\log n)$	≈ 10
$O(n)$	1.000
$O(n \log n)$	≈ 10.000
$O(n^2)$	1.000.000
$O(2^n)$	$\sim 10^{300}$ (impossível computar)
$O(n!)$	maior que o número de átomos do universo

Tabela 2 – Complexidade e operações
 Fonte: Elaborada pelo autor (2025)

“Operações são unidades básicas de computação que compõem um algoritmo, como comparações, atribuições, somas, leitura/escrita de memória e trocas”
 (Cormen *et al.*, 2009).

2 FUNDAMENTOS DA BUSCA

A operação de busca é uma das tarefas mais fundamentais em ciência da computação. Independentemente do tipo de sistema, seja um software de gestão empresarial, uma aplicação científica, um sistema operacional, um algoritmo de inteligência artificial ou um programa simples para manipulação de vetores, quase

sempre surge a necessidade de localizar um determinado elemento dentro de um conjunto de dados.

A busca aparece em atividades triviais, como:

- Procurar um número dentro de um vetor;
- Verificar se um nome está registrado em uma lista;
- Localizar um arquivo no sistema operacional;
- Conferir se uma senha existe em um banco de dados;
- Identificar se um sensor registrou determinado valor;
- Encontrar uma mensagem específica em uma fila de eventos.

Segundo Knuth (1998), o ato de buscar é tão recorrente que pode ser considerado um dos pilares da computação moderna, assim como ordenar, armazenar e transmitir informações. Sem algoritmos eficientes de busca, diversas aplicações seriam inviáveis em escala real, como bancos de dados contendo milhões de registros, mecanismos de busca na internet ou sistemas industriais que monitoram milhares de variáveis por segundo.

Nesse sentido, compreender a lógica por trás dos algoritmos de busca, seus fundamentos matemáticos, limitações e capacidades, é essencial para qualquer profissional de computação. Além disso, a escolha adequada do algoritmo afeta diretamente o desempenho de um sistema, podendo significar a diferença entre uma aplicação eficaz e uma aplicação impraticável.

2.1 O problema da busca

O problema da busca pode ser formulado da seguinte maneira: dado um conjunto de elementos e uma chave de pesquisa, é necessário determinar se a chave aparece no conjunto e, em caso afirmativo, identificar sua posição.

Formalmente:

$$S = \{a_1, a_2, a_3, \dots, a_n\} \text{ chave } k$$

O objetivo é encontrar um índice i tal que $a_i = k$, ou concluir que tal i não existe.

Essa definição simples dá origem a diferentes abordagens, que variam conforme:

- O tipo de estrutura de dados (vetores, listas, árvores ou tabelas *hash*);
- O estado de organização dos dados (ordenados ou desordenados);
- O custo do acesso aos elementos (acesso direto ou sequencial);
- A limitação de tempo e a memória do ambiente.

É por isso que não existe um único "melhor algoritmo de busca" universal, já que a escolha depende do contexto. Para compreender o desempenho de algoritmos de busca, é fundamental interpretar o parâmetro n , que representa o número de elementos no conjunto onde a busca será realizada.

Assim, se o conjunto possui:

- 10 elementos $\rightarrow n = 10$;
- 100.000 elementos $\rightarrow n = 100.000$;
- 5 milhões de registros $\rightarrow n = 5.000.000$.

O tempo de execução da busca depende diretamente de n ou de alguma função associada (como $\log n$, n^2 etc.). Por isso, a análise de complexidade é essencial para compreender o comportamento dos algoritmos de busca.

2.2 Operações fundamentais da busca

A seguir, apresentaremos as operações básicas envolvidas em qualquer algoritmo de busca.

2.2.1 Operação 1 – Acesso a uma posição (*access operation*)

Em vetores, acessar um elemento por índice é uma operação de tempo constante ($O(1)$). Isso ocorre porque o endereço é calculado diretamente pelo hardware:

$$\text{endereço} = \text{base} + i \times \text{tamanho do elemento}$$

```
int v[5] = {10, 20, 30, 40, 50};  
int elemento = v[2]; // acessa diretamente 30
```

Código-fonte 5 – Exemplo de acesso a uma posição em um programa em C
Fonte: Elaborado pelo autor (2025)

2.2.2 Operação 2 – Comparação (*comparison operation*)

Toda busca exige a comparação entre o valor buscado e um elemento da estrutura:

1. $v[i] == \text{chave}$.

Essa é a operação central tanto da busca linear quanto da binária.

```
if (v[i] == chave) {  
    printf("Elemento encontrado!\n");  
}
```

Código-fonte 6 – Exemplo de comparação em um programa em C
Fonte: Elaborado pelo autor (2025)

2.2.3 Operação 3 – Iteração sobre elementos (*iteration operation*)

Essa iteração está presente em buscas sequenciais. É o ato de percorrer cada elemento, normalmente com um laço:

```
for (int i = 0; i < n; i++) {  
    if (v[i] == chave) return i;  
}
```

Código-fonte 7 – Exemplo de busca linear em um programa em C
Fonte: Elaborado pelo autor (2025)

Neste caso, a busca pode executar até n iterações, justificando $O(n)$.

2.2.4 Operação 4 - Divisão do intervalo (*divide operation*)

É uma característica da busca binária. O intervalo é reduzido pela metade a cada comparação.

Visualmente:

[INÍCIO | ... | MEIO | ... | FIM]

Se a chave for menor que o elemento do meio, ignoramos a metade da direita.

```
int meio = (inicio + fim) / 2;

if (chave < v[meio]) {
    fim = meio - 1;      // descarta metade direita
} else {
    inicio = meio + 1;   // descarta metade esquerda
}
```

Código-fonte 8 – Exemplo de um passo fundamental da busca binária em um programa em C
Fonte: Elaborado pelo autor (2025)

Essa divisão recorrente produz a complexidade $O(\log n)$.

2.2.5 Operação 5 – Retorno da posição (*return operation*)

Conclui a busca ao encontrar o elemento ou ao determinar sua ausência.

```
return meio;    // encontrado
// ou
return -1;      // não encontrado
```

Código-fonte 9 – Exemplo de retorno da posição em um programa em C
Fonte: Elaborado pelo autor (2025)

Essa convenção (-1) é amplamente utilizada em C e em bibliotecas clássicas.

2.2.6 Operação 6 – Mudança de ponteiros / referências (*pointer update*)

Em listas encadeadas, o processo de busca exige movimentar o ponteiro para o próximo nó.

```
No *atual = lista;
```

```
while (atual != NULL) {  
    if (atual->valor == chave)  
        return atual;  
  
    atual = atual->prox; // avança o ponteiro  
}  
return NULL;
```

Código-fonte 10 – Exemplo de busca encadeada em um programa em C
Fonte: Elaborado pelo autor (2025)

Essa operação tem custo linear, pois percorre cada nó exatamente uma vez.

2.3 Por que essas operações importam?

Essas operações são importantes pois cada algoritmo de busca é definido pelo conjunto e pela ordem com que utiliza essas operações.

- A busca linear usa: acesso + comparação + iteração $\rightarrow O(n)$;
- A busca binária usa: acesso + comparação + divisão do intervalo $\rightarrow O(\log n)$;
- Busca em listas: comparação + ponteiro + iteração linear $\rightarrow O(n)$;
- Estruturas sofisticadas (como árvores): acesso indireto + comparação + recursão $\rightarrow O(\log n)$ em árvores balanceadas.

Essas operações são a “gramática” sobre a qual algoritmos de busca são construídos.

2.4 Tipos fundamentais de operações de busca

A busca pode ser realizada de formas diferentes, dependendo de como os dados estão organizados. As duas operações fundamentais são:

1. Busca linear (ou sequencial);
2. Busca binária.

A seguir, vamos entender um pouco mais sobre cada uma delas, com explicações detalhadas e um exemplo em C para cada operação.

2.4.1 Busca linear

A busca linear consiste em percorrer o conjunto do primeiro ao último elemento, verificando cada posição até encontrar a chave desejada ou concluir que ela não está presente. Ela não exige qualquer organização prévia dos dados, já que funciona tanto em vetores ordenados quanto desordenados.

Algumas características principais incluem:

- Simples de implementar;
- Funciona em qualquer coleção sequencial;
- Custo proporcional ao tamanho da entrada:

$$T(n) = O(n)$$

- Melhor caso: elemento está na primeira posição $\rightarrow O(1)$;
- Pior caso: elemento não está presente $\rightarrow O(n)$.

Esse fluxo lógico/fluxograma representado pela figura a seguir pode ser resumido em cinco etapas fundamentais:

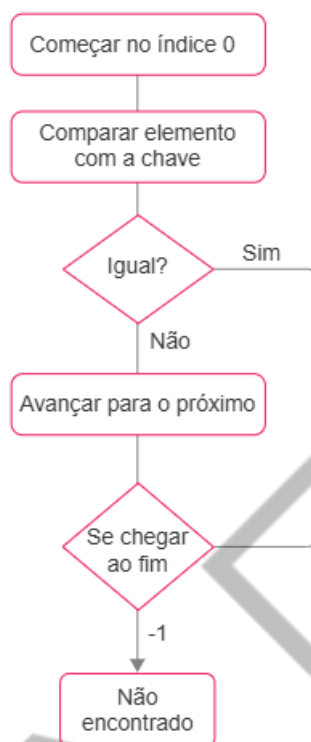


Figura 1 – Fluxograma da busca linear
Fonte: Elaborada pelo autor (2025)

Esse procedimento linear, embora simples, ainda é amplamente utilizado em contextos em que o conjunto de dados é pequeno, em sistemas com recursos restritos ou em situações em que a estrutura não se encontra ordenada e outras técnicas mais eficientes não podem ser aplicadas.

Antes de analisarmos o código em C, é importante compreender como funciona o fluxo lógico, pois esse algoritmo segue uma sequência simples, direta e intuitiva. A busca linear consiste em percorrer os elementos do vetor um a um, iniciando no primeiro índice e avançando progressivamente até o último. Em cada posição, o algoritmo compara o valor armazenado com a chave que deseja encontrar.

Se, em algum momento, ocorre uma igualdade, a busca termina imediatamente e retorna à posição em que o elemento foi encontrado. Caso todos os elementos sejam examinados e nenhum deles corresponda à chave, o algoritmo conclui que o item não existe no vetor e retorna um valor indicativo de falha, geralmente -1 .

Esse comportamento sequencial e determinístico torna a busca linear um dos algoritmos mais acessíveis para estudantes que estão iniciando no estudo de estruturas de dados. Além disso, sua lógica simples permite compreender facilmente como o custo operacional cresce em função do número de elementos, conectando

diretamente com os conceitos de complexidade, especialmente o tempo $O(n)$, que caracteriza algoritmos cujo desempenho aumenta proporcionalmente ao tamanho da entrada.

Com esse fluxo lógico em mente, podemos agora observar como essa sequência de operações é traduzida para um programa em linguagem C.

```
#include <stdio.h>

int buscaLinear(int v[], int n, int chave) {
    for (int i = 0; i < n; i++) {
        if (v[i] == chave) {
            return i; // elemento encontrado
        }
    }
    return -1; // elemento não encontrado
}

int main() {
    int dados[] = {4, 7, 2, 9, 3};
    int n = 5;

    int pos = buscaLinear(dados, n, 9);

    if (pos != -1)
        printf("Elemento encontrado na posicao %d\n", pos);
    else
        printf("Elemento nao encontrado.\n");

    return 0;
}
```

Código-fonte 11 – Exemplo de código da busca linear em um programa em C
Fonte: Elaborado pelo autor (2025)

2.4.2 Busca binária

A busca binária é muito mais eficiente que a busca linear, mas possui um requisito indispensável: a coleção deve estar ordenada. A ideia é dividir o vetor ao meio e reduzir o espaço de busca pela metade a cada comparação. Essa redução iterativa gera uma complexidade assintótica, o que significa um ganho enorme de desempenho quando n é grande:

$$T(n) = O(\log n)$$

O fluxograma da figura abaixo representa, passo a passo, o funcionamento lógico do algoritmo de busca binária, permitindo visualizar graficamente como o processo de busca é conduzido dentro de um vetor ordenado. Cada bloco representa uma operação fundamental necessária para reduzir progressivamente o intervalo de busca, tornando o processo extremamente eficiente.

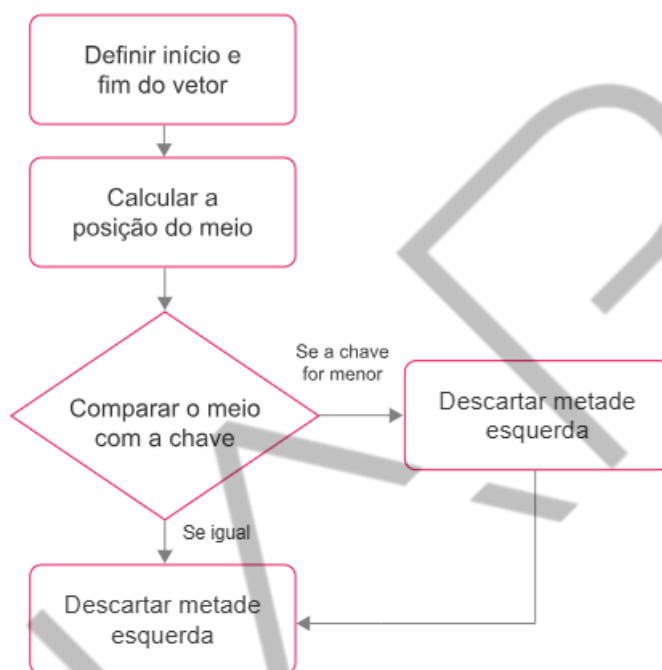


Figura 2 – Fluxograma da busca binária
Fonte: Elaborada pelo autor (2025)

O algoritmo inicia com a definição das extremidades do vetor, representadas pelos índices início e fim. Esses dois valores determinam o intervalo atual em que a busca será realizada. Em seguida, calcula-se a posição do meio, obtida pela média entre início e fim. Esse cálculo é o ponto central do funcionamento da busca binária, pois o elemento localizado nesta posição será comparado diretamente com a chave buscada.

A comparação da chave com o elemento central define o fluxo da operação. Se houver igualdade, a busca é concluída imediatamente e o índice encontrado é retornado. Caso a chave seja menor que o elemento do meio, a metade direita do vetor é descartada, atualizando a variável fim. Por outro lado, se a chave for maior, descarta-se toda a metade esquerda, ajustando-se o valor de início.

Esse processo de eliminação sucessiva continua enquanto ainda houver elementos no intervalo. É essa estratégia de eliminação pela metade que confere ao

algoritmo sua eficiência característica, resultando em uma complexidade de tempo $O(\log n)$. Em termos práticos, mesmo para vetores com milhões de elementos, o número de comparações necessárias permanece extremamente baixo.

A visualização proporcionada pelo fluxograma facilita a compreensão do comportamento interno da busca binária, permitindo estabelecer uma relação direta entre o diagrama e o código em C. Cada bloco do fluxograma corresponde a uma etapa presente na implementação do algoritmo.

Por exemplo:

- Definir início e fim $\rightarrow$ corresponde à inicialização das variáveis $\text{inicio} = 0$ e $\text{fim} = n - 1$;
- Calcular o meio $\rightarrow$ representa a linha $\text{int meio} = (\text{inicio} + \text{fim}) / 2$;
- Comparar a chave $\rightarrow$ é o teste condicional central:
 $\text{if (v[meio] == chave)}$;
- Descartar metade esquerda ou direita $\rightarrow$ decorre dos ajustes $\text{fim} = \text{meio} - 1$; ou $\text{inicio} = \text{meio} + 1$;

A leitura do fluxograma antes da análise do código ajuda a compreender a lógica antes da sintaxe, evitando que a implementação pareça apenas um conjunto de instruções sem propósito. O fluxograma oferece uma visão mais abstrata e conceitual, enquanto o código traduz essa lógica em instruções específicas da linguagem C.

Essa combinação, fluxograma + algoritmo, aprimora o entendimento da técnica de busca binária e facilita a aprendizagem de conceitos como divisão do problema, eliminação de casos impossíveis e raciocínio algorítmico baseado em complexidade logarítmica.

```
#include <stdio.h>

int buscaBinaria(int v[], int n, int chave) {
    int inicio = 0;
    int fim = n - 1;

    while (inicio <= fim) {
        int meio = (inicio + fim) / 2;
```

```
        if (v[meio] == chave) {
            return meio; // elemento encontrado
        }
        else if (chave < v[meio]) {
            fim = meio - 1; // busca à esquerda
        }
        else {
            inicio = meio + 1; // busca à direita
        }
    }

    return -1; // elemento não existe
}

int main() {
    int dados[] = {2, 5, 7, 9, 12, 15, 20};
    int n = 7;

    int pos = buscaBinaria(dados, n, 12);

    if (pos != -1)
        printf("Elemento encontrado na posicao %d\n", pos);
    else
        printf("Elemento nao encontrado.\n");

    return 0;
}
```

Código-fonte 12 – Exemplo de código da busca binária em um programa em C
Fonte: Elaborado pelo autor (2025)

A partir da compreensão dos fundamentos da busca, torna-se evidente que a eficiência de um algoritmo de pesquisa depende não apenas da técnica aplicada, mas também da organização dos dados. Observou-se, por exemplo, que, enquanto a busca linear trabalha adequadamente em conjuntos desordenados, a busca binária só se torna possível quando os elementos estão previamente ordenados. Essa dependência da ordenação não é incidental, mas sim um reflexo direto da forma como a estrutura dos dados influencia o custo computacional das operações.

Em outras palavras, o estudo de algoritmos de busca revela um princípio essencial: “quanto melhor estruturados e organizados forem os dados, mais eficientes podem ser os algoritmos que operam sobre eles” (Cormen *et al.*, 2009).

Assim, a busca e ordenação não são tópicos independentes. Pelo contrário, constituem um par complementar na ciência da computação. A ordenação prepara o

terreno para buscas mais rápidas, enquanto a busca, por sua vez, motiva a necessidade de reorganizar dados para reduzir o tempo de acesso.

Além disso, diversos sistemas computacionais, como bancos de dados, sistemas de arquivos, indexadores de busca, *pipelines* de dados em sistemas embarcados e estruturas avançadas como árvores balanceadas, dependem tanto da ordenação quanto da busca para operar de maneira eficiente. Muitas aplicações só alcançam desempenho adequado graças a algoritmos de ordenação que servem como pré-processamento para buscas subsequentes.

A transição natural, portanto, consiste em compreender que:

- A busca binária deriva diretamente da ordenação;
- O desempenho de inúmeras operações subsequentes depende da forma como os dados foram ordenados;
- A escolha do algoritmo de ordenação impacta toda a cadeia de processamento.

Dessa forma, concluído o estudo das técnicas fundamentais de busca, avançamos agora para os algoritmos responsáveis por ordenar coleções de dados, entendendo seus princípios, comportamentos e custos computacionais, de modo a consolidar a base necessária para estruturas e algoritmos mais sofisticados.

3 FUNDAMENTOS DA ORDENAÇÃO

A ordenação é uma das tarefas mais estudadas na ciência da computação, não apenas pelo seu valor prático, mas também porque revela princípios fundamentais de eficiência, estruturação e manipulação de dados.

Ordenar consiste em reorganizar os elementos de uma coleção de acordo com algum critério, geralmente em ordem crescente ou decrescente. A princípio, esse conceito parece simples, mas sua importância cresce drasticamente quando consideramos grandes volumes de dados ou operações que dependem diretamente dessa organização.

Como destacado por Knuth (1998), a ordenação está no centro de inúmeros sistemas: algoritmos de busca eficientes, mecanismos de indexação em bancos de dados, compressão, redes de computadores, estatística computacional, inteligência artificial e sistemas operacionais. A busca binária, inclusive, só é possível quando os dados estão previamente ordenados.

3.1 Por que ordenar? A motivação computacional

Ordenar é uma operação com diversas finalidades, como:

- Melhorar a eficiência de busca, habilitando métodos como a busca binária;
- Organizar dados para apresentação, como listas de alunos ou tabelas de vendas;
- Preparar dados para algoritmos avançados, como divisão e conquista, compressão ou classificação inteligente;
- Facilitar análises estatísticas, como cálculo de medianas, quartis ou distribuições;
- Simplificar comparações, detecção de duplicatas e verificações de consistência.

Em suma, a ordenação reduz o custo de várias operações posteriores e abre a porta para algoritmos mais eficientes.

3.2 Classificação dos algoritmos de ordenação

Apesar de existirem dezenas de métodos diferentes, os algoritmos de ordenação são normalmente agrupados em três categorias:

1. **Ordenações simples ($O(n^2)$):** adequadas para vetores pequenos, pela simplicidade de implementação:
 - Bubble sort;
 - Selection sort;

- Insertion sort.

Apesar de ineficientes em grandes volumes, são úteis em introdução à programação e ambientes embarcados simples.

2. Ordenações intermediárias: comportamentos melhores em casos específicos:

- Shell sort;
- Comb sort.

Costumam apresentar bom desempenho prático em vetores médios.

3. Ordenações eficientes ($O(n \log n)$): fundamentais na computação moderna:

- Merge sort;
- Quick sort;
- Heap sort.

Alguns algoritmos são estáveis, outros não; alguns consomem memória extra, outros trabalham *in-place*. Portanto, cada um atende um cenário diferente.

3.3 Ordenação simples

Os algoritmos de ordenação simples, também chamados de ordenações elementares, são técnicas introdutórias cujo foco está menos na eficiência e mais na clareza conceitual. Apesar de apresentarem desempenho limitado para grandes conjuntos de dados, esses algoritmos desempenham um importante papel pedagógico: eles ajudam na compreensão de operações fundamentais como comparação, permutação, varredura sequencial, estabilização e controle de laços aninhados.

Métodos como *bubble sort*, *insertion sort* e *selection sort* ilustram de maneira intuitiva como a manipulação ordenada de elementos ocorre. É a partir desses princípios básicos que algoritmos mais sofisticados, como *merge sort* e *quick sort*,

evoluem. Por isso, iniciamos o estudo das ordenações justamente por esses métodos simples e conceitualmente acessíveis.

3.3.1 Bubble sort

É o mais didático dos algoritmos e sua lógica imita o processo de “borbulhar” os maiores valores para o final do vetor a cada passagem.

Funcionamento:

- Percorrer o vetor repetidamente;
- Comparar pares consecutivos;
- Trocar se estiverem fora de ordem.

A figura abaixo apresenta de forma ilustrada o funcionamento da ordenação *bubble sort* para facilitar o entendimento de todas as etapas.

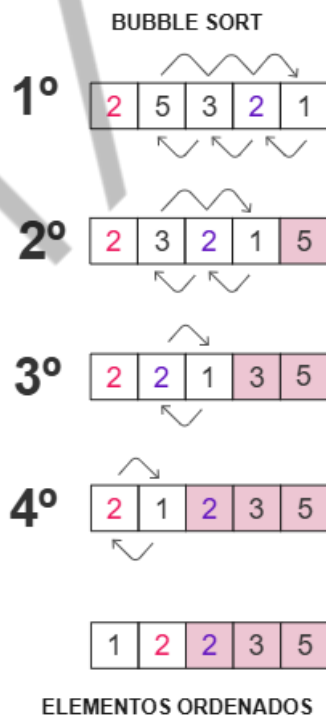


Figura 3 – Funcionamento do *bubble sort*
Fonte: Adaptada de Rego (2023)

Abaixo segue o código em C para implementação da ordenação *bubble sort*.

```
// Bubble Sort - Ordenação simples por troca
void bubbleSort(int v[], int n) {
    int i, j, temp;

    // Loop externo controla o número de passagens no vetor
    for (i = 0; i < n - 1; i++) {

        // Loop interno compara elementos adjacentes
        for (j = 0; j < n - 1 - i; j++) {

            // Se o elemento atual é maior que o próximo,
            trocamos
            if (v[j] > v[j + 1]) {
                temp = v[j];
                v[j] = v[j + 1];
                v[j + 1] = temp;
            }
        }
    }
}
```

Código-fonte 13 – Exemplo de código de ordenação *bubble sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

Pontos-chave:

1. No `for (i = 0; i < n - 1; i++)`: cada passagem coloca o maior elemento restante no final;
2. No `for (j = 0; j < n - 1 - i; j++)`: a cada passagem, o final do vetor já está ordenado;
3. A troca é feita com uma variável auxiliar `temp`.

3.3.2 Selection sort

Baseia-se em seleccionar o menor elemento do vetor e colocá-lo na posição correta a cada iteração. A figura a seguir apresenta o funcionamento da ordenação feita pelo algoritmo *selection sort* e, logo abaixo, o código para implementação em C.

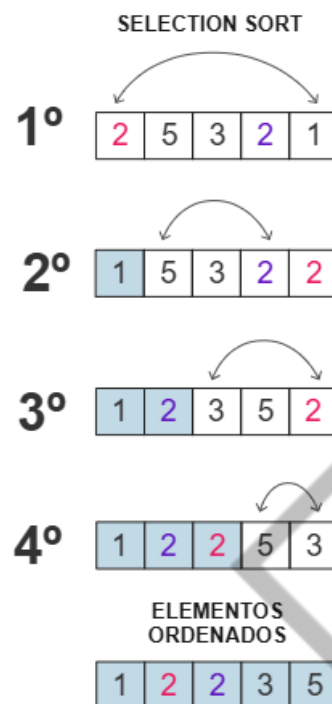


Figura 4 – Funcionamento do *selection sort*
Fonte: Adaptado de Rego (2023)

```
// Selection Sort - Seleciona o menor elemento e move para a
// posição correta
void selectionSort(int v[], int n) {
    int i, j, minIndex, temp;

    // A cada iteração escolhemos a posição i para colocar o
    // menor elemento
    for (i = 0; i < n - 1; i++) {
        minIndex = i; // assumimos que o menor está em i

        // Procuramos o menor elemento no restante do vetor
        for (j = i + 1; j < n; j++) {
            if (v[j] < v[minIndex]) {
                minIndex = j;
            }
        }

        // Após encontrar o menor elemento, trocamos com v[i]
        temp = v[i];
        v[i] = v[minIndex];
        v[minIndex] = temp;
    }
}
```

Código-fonte 14 – Exemplo de código de ordenação *selection sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

Pontos-chave:

1. O algoritmo faz poucas trocas, porém muitas comparações;
2. Muito útil quando trocas são mais custosas que comparações.

3.3.3 Insertion sort

Simula a forma como organizamos cartas na mão, inserindo cada elemento na posição correta dentro da parte já ordenada.

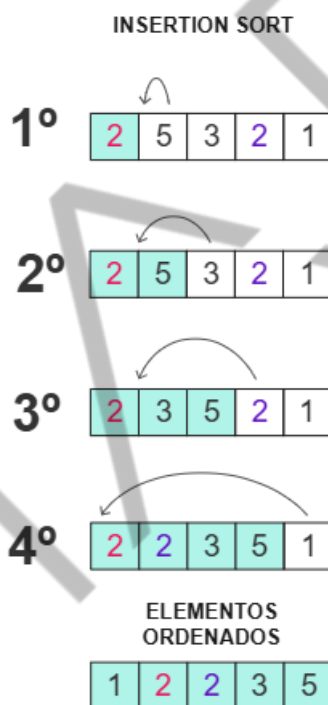


Figura 5 – Funcionamento do *insertion sort*
Fonte: Adaptado de Rego (2023)

```
// Insertion Sort - Insere elementos na parte já ordenada do vetor
void insertionSort(int v[], int n) {
    int i, chave, j;

    // começamos do segundo elemento
    for (i = 1; i < n; i++) {
        chave = v[i];      // elemento que será inserido
        j = i - 1;
    }
}
```

```
// deslocamos elementos maiores que a chave para a
direita
while (j >= 0 && v[j] > chave) {
    v[j + 1] = v[j];
    j--;
}

// colocamos a chave na posição correta
v[j + 1] = chave;
}
```

Código-fonte 15 – Exemplo de código de ordenação *insertion sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

Pontos-chave:

1. Muito eficiente para listas quase ordenadas;
2. Utiliza deslocamento ao invés de troca;
3. É o algoritmo utilizado pelo Python em pequenos subconjuntos (*TimSort hybrid*).

3.4 Ordenação intermediária

Os algoritmos de ordenação intermediária situam-se entre os métodos simples, como *bubble sort*, *selection sort* e *insertion sort*, e os métodos avançados, como *merge sort* e *quick sort*. Eles surgem como alternativas que exploram melhor algumas propriedades das entradas, reduzindo o número de comparações ou o número de movimentações de elementos.

Diferentemente dos algoritmos simples, que possuem complexidade quadrática $O(n^2)$, mas são fáceis de implementar, e dos algoritmos avançados, geralmente $O(n \log n)$, mas dependentes de estruturas auxiliares ou recursão, os algoritmos intermediários tentam atingir um equilíbrio interessante:

- Possuem desempenho superior aos quadráticos em casos reais;
- Não exigem estruturas complexas ou maior uso de memória.

Eles são especialmente úteis em situações onde:

- Os dados possuem algum nível de ordenação prévia;
- Deseja-se reduzir o número de movimentações em relação ao *insertion sort*;
- Busca-se um algoritmo mais rápido que os quadráticos, mas sem a complexidade de implementação dos métodos avançados.

Os dois algoritmos mais representativos dessa categoria são:

1. Shell sort;
2. Comb sort.

3.4.1 Shell sort

O *shell sort*, proposto originalmente por Donald Shell em 1959, é uma generalização do *insertion sort*. Sua ideia principal é permitir a comparação de elementos distantes, e não apenas vizinhos, diminuindo drasticamente o número de movimentos necessários para “puxar” valores grandes ou pequenos para perto de suas posições finais. No *insertion sort*, um elemento muito pequeno localizado no final do vetor precisa ser deslocado posição por posição até o início, custando $O(n)$ movimentos individuais. Isso contribui para sua complexidade quadrática.

O *shell sort* corrige esse problema introduzindo *gaps* (saltos) entre os elementos comparados. A seguir temos as situações que podemos utilizá-lo:

- Vetores de tamanho médio (até dezenas de milhares);
- Quando se deseja algo mais rápido que *insertion sort*, sem recursão;
- Em ambientes embarcados onde a memória extra é limitada.

Abaixo temos o código comentado na linguagem em C do algoritmo.

```
#include <stdio.h>

void shellSort(int v[], int n) {
    // Começamos com um gap grande, reduzindo aos poucos
    for (int gap = n / 2; gap > 0; gap /= 2) {
```

```
// Aplicamos Insertion Sort modificado para cada gap
for (int i = gap; i < n; i++) {
    int temp = v[i];
    int j = i;

    // Movemos elementos distantes 'gap' posições
    while (j >= gap && v[j - gap] > temp) {
        v[j] = v[j - gap];
        j -= gap;
    }

    // Inserimos o elemento na posição correta
    v[j] = temp;
}
}
```

Código-fonte 16 – Exemplo de código de ordenação *shell sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

3.4.2 Comb sort

O *comb sort* é uma evolução do *bubble sort*. Seu objetivo é resolver o problema dos “coelhos e tartarugas”:

- Coelhos: elementos muito grandes no início que rapidamente “saltam”;
- Tartarugas: elementos muito pequenos no final que demoram a se mover.

O *comb sort* acelera a movimentação dessas tartarugas, eliminando-as rapidamente usando *gaps* semelhantes ao *shell sort*, porém com uma regra específica de redução. Podemos utilizá-lo:

- Como substituto direto do *bubble sort*;
- Para vetores médios em que se deseja simplicidade e rapidez;
- Em situações onde não há a necessidade de recursão ou estruturas extras.

```
#include <stdio.h>

int getNextGap(int gap) {
    // Fator de redução recomendado: 1.3
```



```
    gap = (gap * 10) / 13;
    if (gap < 1) return 1;
    return gap;
}

void combSort(int v[], int n) {
    int gap = n;
    int trocou = 1;

    while (gap != 1 || trocou) {
        gap = getNextGap(gap);
        trocou = 0;

        for (int i = 0; i + gap < n; i++) {
            // Compara elementos distantes 'gap'
            if (v[i] > v[i + gap]) {
                int temp = v[i];
                v[i] = v[i + gap];
                v[i + gap] = temp;
                trocou = 1;
            }
        }
    }
}
```

Código-fonte 17 – Exemplo de código de ordenação *comb sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

3.5. Algoritmos de ordenação eficiente ($O(n \log n)$)

Embora os métodos simples e intermediários de ordenação sejam úteis em situações específicas, especialmente quando a entrada é pequena, parcialmente ordenada ou quando se prioriza a simplicidade do código, aplicações reais de grande escala demandam algoritmos mais sofisticados. Esses algoritmos alcançam desempenho assintótico na ordem de $O(n \log n)$, considerado o melhor para algoritmos baseados em comparação, conforme demonstrado por Knuth (1998). Em contextos como sistemas de banco de dados, renderização gráfica, análise de grandes conjuntos de dados, indexação e sistemas operacionais, métodos eficientes são indispensáveis.

Esse grupo de algoritmos explora estratégias como divisão e conquista, árvores de *heaps*, particionamento recursivo e estruturas auxiliares para reduzir o número de comparações necessárias. Os três algoritmos mais representativos, *merge sort*, *quick*

sort e *heap sort*, serão apresentados em detalhes a seguir, com explicações conceituais, exemplos, análises de complexidade e implementações comentadas em linguagem C.

3.5.1 Merge sort

O *merge sort* é um dos algoritmos mais elegantes e estáveis da família “dividir para conquistar”. Sua essência consiste em dividir recursivamente o vetor em dois subvetores menores, ordená-los independentemente e, por fim, intercalar (merge) os resultados de forma ordenada. A ideia de forma geral segue o seguinte padrão:

1. Dividir o vetor ao meio até sobrar apenas um elemento (caso base);
2. Ordenar cada metade recursivamente;
3. Intercalar as duas metades já ordenadas.

A etapa de intercalação é linear, e o número de níveis da recursão é logarítmico, produzindo o desempenho:

$$T(n) = O(n \log n)$$

O código para essa ordenação fica da seguinte maneira:

```
// Função auxiliar para intercalar duas metades ordenadas
void merge(int v[], int inicio, int meio, int fim) {
    int i = inicio, j = meio + 1, k = 0;
    int aux[fim - inicio + 1]; // vetor temporário

    // Intercala enquanto houver elementos nas duas metades
    while (i <= meio && j <= fim) {
        if (v[i] <= v[j]) {
            aux[k++] = v[i++];
        } else {
            aux[k++] = v[j++];
        }
    }

    // Copia o restante da primeira metade, se existir
    while (i <= meio)
        aux[k++] = v[i++];
}
```

```
// Copia o restante da segunda metade
while (j <= fim)
    aux[k++] = v[j++];

// Copia tudo de volta ao vetor original
for (i = inicio, k = 0; i <= fim; i++, k++)
    v[i] = aux[k];
}

// Merge Sort recursivo
void mergeSort(int v[], int inicio, int fim) {
    if (inicio < fim) { // caso base: vetor de tamanho 1
        int meio = (inicio + fim) / 2;

        mergeSort(v, inicio, meio);           // ordena primeira
metade
        mergeSort(v, meio + 1, fim);          // ordena segunda metade
        merge(v, inicio, meio, fim);          // intercala
    }
}
```

Código-fonte 18 – Exemplo de código de ordenação *merge sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

3.5.2 Quick sort

O *quick sort* é amplamente utilizado devido ao seu desempenho excepcional no caso médio e ao fato de trabalhar *in-place*, ou seja, sem a necessidade de memória auxiliar significativa. Proposto por Tony Hoare em 1960, ele é um dos algoritmos mais adotados em bibliotecas modernas. A complexidade média é semelhante à da *merge sort*. A ideia do funcionamento é feita da seguinte maneira:

1. Escolhe-se um pivô;
2. Particiona-se o vetor em duas partes:
 - a. Elementos menores que o pivô;
 - b. Elementos maiores que o pivô.
3. Ordena-se cada partição recursivamente.

O código para essa ordenação está representado abaixo.

```
// Função de partição: organiza os elementos ao redor do pivô
int particiona(int v[], int inicio, int fim) {
```

```
int pivo = v[fim]; // pivô escolhido
int i = inicio - 1;

for (int j = inicio; j < fim; j++) {
    if (v[j] <= pivo) { // move menores para a esquerda
        i++;
        int temp = v[i];
        v[i] = v[j];
        v[j] = temp;
    }
}

// Coloca o pivô na posição correta
int temp = v[i + 1];
v[i + 1] = v[fim];
v[fim] = temp;

return i + 1; // retorna a posição do pivô
}

// Quick Sort recursivo
void quickSort(int v[], int inicio, int fim) {
    if (inicio < fim) {
        int p = particiona(v, inicio, fim); // posição do pivô
        quickSort(v, inicio, p - 1);       // ordena parte
        esquerda
        quickSort(v, p + 1, fim);           // ordena parte
        direita
    }
}
```

Código-fonte 19 – Exemplo de código de ordenação *quick sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

3.5.3 Heap sort

O *heap sort* utiliza uma estrutura auxiliar chamada *heap* binário, uma árvore quase completa em que cada pai é maior (*heap* máximo) ou menor (*heap* mínimo) do que seus filhos. A ordenação ocorre transformando o vetor em um *heap* e repetidamente extraindo o maior elemento. A principal ideia de funcionamento abrange:

1. Construir um *heap* máximo;
2. Trocar o maior elemento (raiz) com a última posição do vetor;

3. Reduzir o tamanho do *heap* e reestabelecer a propriedade de *heap*;
4. Repetir até ordenar todos os elementos.

O código para essa ordenação fica da seguinte maneira:

```
// Função para transformar o vetor em um heap máximo
void heapify(int v[], int n, int i) {
    int maior = i; // raiz
    int esq = 2 * i + 1; // filho esquerdo
    int dir = 2 * i + 2; // filho direito

    // Verifica se filho esquerdo é maior
    if (esq < n && v[esq] > v[maior])
        maior = esq;

    // Verifica se filho direito é maior
    if (dir < n && v[dir] > v[maior])
        maior = dir;

    // Se maior não for a raiz, troca e continua ajustando
    if (maior != i) {
        int temp = v[i];
        v[i] = v[maior];
        v[maior] = temp;

        heapify(v, n, maior); // recursão
    }
}

// Heap Sort completo
void heapSort(int v[], int n) {
    // Construção do heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(v, n, i);

    // Extração repetida do maior
    for (int i = n - 1; i > 0; i--) {
        int temp = v[0];
        v[0] = v[i];
        v[i] = temp;

        heapify(v, i, 0); // reconstrói heap
    }
}
```

Código-fonte 20 – Exemplo de código de ordenação *heap sort* em um programa em C
Fonte: Elaborado pelo autor (2025)

4 CONCLUSÃO

Ao longo deste capítulo, percorremos um conjunto fundamental de conceitos da ciência da computação: a análise de complexidade, os algoritmos de busca e os diversos métodos de ordenação. Cada parte contribuiu para formar uma base sólida capaz de sustentar o raciocínio lógico, matemático e computacional necessário para o desenvolvimento de algoritmos eficientes.

Iniciamos discutindo complexidade de algoritmos, estabelecendo a importância do parâmetro n , o tamanho da entrada, como elemento central para compreender desempenho, escalabilidade e limitações computacionais. A introdução à notação Big-O permitiu avaliar, de maneira abstrata e independente da máquina, como algoritmos se comportam à medida que crescem em volume de dados. Essa fundamentação teórica foi essencial para compreender tanto os algoritmos de busca quanto os mecanismos de ordenação.

Em seguida, adentramos os fundamentos da busca, analisando desde a simplicidade da busca linear, com seu comportamento $O(n)$, até a eficiência exponencialmente superior da busca binária, cuja complexidade $O(\log n)$ só é possível graças à pré-condição da ordenação. Essa relação direta entre busca e ordenação mostrou um ponto central: estruturas ordenadas habilitam algoritmos muito mais rápidos. Com isso, tornou-se claro por que sistemas modernos priorizam manter dados ordenados, indexados e organizados.

Na sequência, estudamos as técnicas de ordenação, inicialmente observando as ordenações simples (*bubble sort*, *selection sort* e *insertion sort*). Esses algoritmos apresentam conceitos fundamentais para iniciantes, mas sofrem com limitações de desempenho, geralmente $O(n^2)$. Apesar disso, possuem grande valor didático e aplicabilidade em conjuntos pequenos ou em contextos que exigem estabilidade ou baixo custo de implementação.

Avançamos para os métodos intermediários, como *shell sort* e *comb sort*, que representam tentativas de melhorar o desempenho de algoritmos quadráticos, introduzindo ideias como *gaps* reduzidos e comparações espaçadas. Esses algoritmos são transições importantes entre conceitos básicos e técnicas mais robustas.

Chegamos então aos algoritmos eficientes de ordenação, como *merge sort*, *quick sort* e *heap sort*. Cada um deles apresenta características próprias:

- *Merge sort* ($O(n \log n)$): estável, previsível e muito utilizado em sistemas que manipulam grandes volumes de dados, especialmente onde temos facilidade para alocação adicional de memória;
- *Quick sort* ($O(n \log n)$ no caso médio): extremamente rápido na prática e muito utilizado em implementações reais de bibliotecas padrão, embora apresente um pior caso $O(n^2)$ sob pivô mal escolhido;
- *Heap sort* ($O(n \log n)$): faz uso direto de estruturas de *heap* e garante bom desempenho sem a necessidade de memória extra.

Esses algoritmos demonstraram que o design de soluções eficientes envolve equilíbrio entre desempenho, uso de memória, estabilidade, simplicidade e comportamento.

Por fim, ao conectar busca e ordenação, entendemos que a capacidade de buscar rapidamente depende, muitas vezes, de ordenar de forma correta. Banco de dados, mecanismos de busca, sistemas operacionais, bibliotecas padrão e API's modernas são construídos sobre essa interdependência: dados ordenados permitem buscas logarítmicas, estruturas balanceadas garantem acessos eficientes e algoritmos de ordenação robustos sustentam toda a infraestrutura de software contemporânea.

4.1 Outras ordenações relevantes

Além dos algoritmos estudados neste capítulo, destacam-se outros métodos de grande importância teórica e prática:

- *Radix sort*: ordenação baseada em dígitos (não baseada em comparação), podendo atingir $O(n)$;
- *Counting sort*: muito eficiente para faixas reduzidas de valores; também $O(n)$;

- *Bucket sort*: eficaz para distribuições uniformes e usado em *pipelines* de ordenação;
- *TimSort*: algoritmo híbrido utilizado no Python e Java, derivado de *merge sort* + *insertion sort*;
- *IntroSort*: algoritmo adaptativo que combina *quick sort* + *heap sort*, usado no C++ STL.

Essas técnicas, embora não abordadas em profundidade nesse momento, complementam o repertório do programador e demonstram a contínua evolução do campo de algoritmos.

Dominar busca e ordenação não é apenas um exercício acadêmico; é um dos pilares da formação em computação. Assim, concluímos esse conteúdo com a compreensão de que todo algoritmo eficiente nasce de boas decisões estruturais, apoiadas na análise formal, na observação empírica e no entendimento profundo das relações entre dados, operações e complexidade.

REFERÊNCIAS

ASCENCIO, A. F.; CAMPOS, E. A. V. *Fundamentos da Programação de Computadores*. São Paulo: Pearson, 2012.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. MIT Press, 2009.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. *Lógica de Programação*. 4. ed. São Paulo: Prentice Hall, 2005.

KNUTH, D. E. *The Art of Computer Programming* – Volume 1: Fundamental Algorithms. Addison-Wesley, 1997.

REGO, A. S. *Algoritmos de Ordenação: Resumo para o Concurso BB (TI)*. 2023. Disponível em: <https://www.estrategiaconcursos.com.br/blog/resumo-algoritmos-de-ordenacao-bb-ti/>. Acesso em: 01 de dez. de 2025

SEEDGEWICK, R.; WAYNE, K. *Algorithms*. Addison-Wesley, 2011.

WEISS, M. A. *Data Structures and Algorithm Analysis in C*. 2. ed. Pearson, 2014.

ZIVIANI, N. *Projeto de Algoritmos*. Cengage Learning, 2011.